1. What am I making?

RADIX ProofPath helps a candidate understand job readiness. Five teammates own five parts: JD Analytics -> Resume Parsing -> Profile Builder -> Talent Check -> Skill Matching. You own part 1.

Turn a job advert into clear requirements.

Help a student see what a job actually asks for. A JD is a job description. Your part reads it, lists the skills and keeps the words that support each requirement.

Why your part matters

Skill Matching cannot make a fair comparison until it knows which skills are required, which are preferred and which are alternatives.

What the user gives you

A job description as PDF, Word (.docx) or pasted text.

What you return

A saved, reviewed list of skills, categories and requirement groups, linked to the original text.

A small example

An advert says Required: Python. The result contains Python, category COD (Coding), the quote Required: Python and a mandatory group. It does not invent a proficiency level.

Start with the working code

This is a working starting point, not an instruction to rebuild an app. Clone the WHOLE repository so shared support is available. Work mainly in jd_analytics/. Read this one guide, run it, then improve a problem you can demonstrate.

Your folder has this guide as PDF and Markdown, backend/, frontend/, tests/, samples/ and run.ps1. Shared support is stored once at the root. The Markdown contains the same teaching content and full examples as this PDF.

2. What exactly must I build?

Organizer requirements - keep these working

- Read any text-based PDF or DOCX JD and return a structured skill list, not just a summary.
- Use LLM extraction: an LLM is a language model that can interpret prose. Keep the existing Groq path; a keyword-only demo does not verify this requirement.
- Place each skill in exactly one of the twelve RADIX categories, or OTHER when none fits. Keep named technologies such as Python.
- Run on its own with at least two different JDs and send the saved result to Module 5. Test the official examples when organizers supply them.

Already implemented

Upload and paste, PDF/Word reading, direct Groq structured extraction, checked quotes, skill editing, required/preferred groups, AND/OR alternatives, saved revisions and a review screen already work. Groq is implemented and mock-tested, not live-verified.

Your development checklist

- 1. Run the existing part before editing it. Use the tiny Python sample and a different JD from data/synthetic/reconstructed/.
- 2. Compare every extracted skill and group with the document. Correct omissions through the review screen; investigate the parser only when you can reproduce the error.
- 3. Improve an actual weakness, such as bare Testing or unfamiliar terms, and add a test using that exact wording. Keep false positives visible too.
- 4. Verify save/reload and the handoff into matching. If you have an authorized Groq key, test a synthetic document with explicit consent and record real model quality separately.

Currently missing or unverified

Official organizer JDs and live Groq evaluation are pending. Scanned image PDFs have no OCR (image-to-text reading) here. Do not rebuild the functioning upload or scoring system.

Extra ProofPath features

ProofPath adds importance, AND/OR groups and source links. These are team extensions. Preserve them; do not describe team weights as an employer's official scoring policy.

3. How does it work?

1. Choose input	**2. Read text**	**3. Find requirements**
4. Check evidence	**5. Review and save**	**6. Hand off**

- 1. Choose input: The user uploads PDF/Word or pastes the full JD. Pasted text is also kept as a private source document.
- 2. Read text: The shared reader extracts page text. Empty, broken or unsupported files produce an error.
- 3. Find requirements: With a configured key and this input's permission, Groq proposes structured fields. Otherwise limited local rules run and the screen says so.
- 4. Check evidence: The server checks each proposed quote against the supplied text. A language model cannot create a new source or a skill rating.
- 5. Review and save: The user edits names, categories, importance and alternatives, then saves a reviewed revision.
- 6. Hand off: Module 5 loads this saved JD and its current revision. Later edits require a new comparison.

Words you will see

Backend = Python server code. Frontend = React browser screen. API = an address the screen calls to send/receive data. A contract/schema is the agreed field list, defined in shared/contracts/models.py. An LLM is a language model that interprets text; Groq runs it. Supabase is the cloud database service.

The shared RADIX categories

COD Coding; DSA Data Structures & Algorithms; OOD Object-Oriented Design; APTI Aptitude; COMM Communication; AI Artificial Intelligence; CLOUD Cloud; SQL SQL; SWE Software Engineering; SYSD System Design; NETW Networking; OS Operating Systems.

Each skill has exactly one category, or OTHER. Descriptive skill families are extra UI filters, not new rated categories. Unknown competency is null, never zero. Confidence describes extraction support, not ability. What-if changes never save a real profile.

4. Inputs and outputs

A candidate supplies the file; Module 5 receives the reviewed Extraction result.

POST /api/jd/extract

The full request example describes a file upload. It is a description of the form fields, not a JSON body to send to this endpoint. The browser sends the actual file as multipart/form-data: a form that can carry a file.

Complete input example: jd_analytics/samples/input.json

```
{
  "method": "POST",
  "path": "/api/jd/extract",
  "encoding": "multipart/form-data",
  "file": "practice-jd-SYNTHETIC.docx",
  "hosted_consent": false
}
```

Important fields

- file / hosted_consent: The file is required. Consent defaults to false; true allows this input's text to reach the configured provider.
- id / revision / document_id: Server-created record ID, saved version and private source ID. Example IDs are snapshots; use new IDs returned by your own run.
- skills / requirement_groups: Skill objects contain names, category codes, quotes and source links. Groups refer to skill_id values and say AND/OR and mandatory/preferred/unspecified.
- review_status / llm_status: needs_review means a human must check it. rules_only means local rules; processed means the model returned accepted output. Neither proves semantic accuracy.

Paste uses POST /api/jd/extract-text with required text (15-120000 characters), optional source_name and hosted_consent=false. GET /api/jd/{id} loads it; PATCH /api/jd/{id} sends the complete edited Extraction, retaining id and revision. Save with review_status=reviewed. A stale save returns 409, meaning reload before retrying.

The next page(s) show the COMPLETE captured response from the existing API, including defaults and empty fields. Long lines wrap for reading. Use samples/output.json for exact machine-readable JSON. IDs, receipt IDs, source hashes and times can differ between runs; do not hardcode them. JSON null means unknown/not present; [] means an empty list.

Complete output example

jd_analytics/samples/output.json


```
{
  "schema_version": "1.0",
  "id": "64c38451-b387-4183-be2e-98f1fa0298be",
  "revision": 1,
  "source_type": "jd",
  "source_file": "practice-jd-SYNTHETIC.docx",
  "company": "Synthetic Teaching Co",
  "role": "Junior Developer",
  "skills": [
    {"skill_id": "48ad0190-b628-49ad-a6ce-6cbcbb11fe55", "skill_name": "Python",
  "category_code": "COD", "skill_family": "Programming Languages", "evidence": "Required:
   Python", "confidence": "medium", "source_page": null, "source_file": "practice-jd-
  SYNTHETIC.docx", "source_document_id": "faad38b4-fea8-429c-98e3-b312464cdae9",
  "source_type": "document", "evidence_type": "jd_requirement", "verification_status":
  "unverified", "claim_status": "claimed", "claim_basis": "explicit", "normalized_name":
  null, "source_start": null, "source_end": null}
  ],
  "requirement_groups": [
    {"requirement_id": "610c96bc-5506-4713-a14f-e7e2828b31ff", "skill_ids":
  ["48ad0190-b628-49ad-a6ce-6cbcbb11fe55"], "operator": "AND", "requirement_type":
  "mandatory", "requires_review": false, "evidence": "Required: Python", "source_page":
  null}
  ],
  "name": "",
  "email": "",
  "education": [],
  "projects": [],
  "experience": [],
  "certifications": [],
  "internships": [],
  "hackathons": [],
  "achievements": [],
  "sections": [
    {"page": null, "text": "SYNTHETIC TEACHING JD - NOT OFFICIAL ORGANIZER DATA\nCompany:
   Synthetic Teaching Co\nRole: Junior Developer\nRequired: Python"}
  ],
  "review_status": "needs_review",
  "extraction_mode": "local_limited",
  "parser_version": "local-layout-review-1.3",
  "input_method": "file",
  "model_version": null,
  "prompt_version": null,
  "llm_status": "rules_only",
  "llm_metadata": {},
  "warnings": ["Limited local vocabulary extraction; implicit requirements and unfamiliar
   technologies need manual review."],
  "document_id": "faad38b4-fea8-429c-98e3-b312464cdae9"
}
```

5. How do I work on the code?

Install and run - Windows PowerShell

Install Python 3.10+, Node.js 24 LTS (includes npm), Git and an editor such as VS Code. Accept the installers' PATH option, reopen PowerShell and clone the full GitHub repository. Open a terminal at the repository root - the folder containing package.json.

```
.\jd_analytics\run.ps1 setup
# Terminal 1:
.\jd_analytics\run.ps1 api
# A SECOND terminal, same root:
.\jd_analytics\run.ps1 web
```

Open http://127.0.0.1:5173/. Server API help is at http://127.0.0.1:8000/docs. setup creates .venv and installs locked Python/npm dependencies plus Chromium for browser tests. Run setup once; no key is needed. Keep api and web running; Ctrl+C stops your own server. Only one module uses 8000/5173 at a time.

Open these files first

- jd_analytics/backend/service.py - Start here. Finds requirements, groups and local parsing warnings; supplies the JD instructions to Groq.
- jd_analytics/backend/router.py - Receives pasted input and joins the upload/review routes. A route is the server address that handles a request.
- jd_analytics/frontend/JDAnalyticsPage.tsx - Owns upload, extraction and save actions on the screen.
- jd_analytics/frontend/RequirementReview.tsx - Edits importance, alternatives and group review; PasteJD.tsx owns pasted input.

shared/integrations/llm.py (Groq calls), shared/integrations/extraction.py (grounding/local skills), shared/integrations/documents.py (file text), shared/contracts/models.py (agreed fields). Read these when needed; coordinate shared changes with the Scrum Master.

Make one useful change

Read a failing example, locate the responsible function, change that part and add a test in your tests/ or frontend test file. Do not rename shared fields, duplicate common code or rewrite other modules. Ask the Scrum Master before changing shared contracts, storage or provider code.

What should I see?

Expect an Upload file / Paste text choice, editable company/role, skill cards with source links, requirement groups and Save reviewed JD. Try a source link before presenting the result.

Load synthetic inputs supplies pre-reviewed Mira records for independent operation. It is not fresh extraction. The tiny Asha terminal examples and the richer Mira browser examples are intentionally different inputs.

6. How do I know it works?

```
.\jd_analytics\run.ps1 example
.\jd_analytics\run.ps1 test
```

example runs the small teaching case in a fresh temporary database and checks its result. test runs your real standalone API check, Python tests, React tests, type checking, lint, your production build and your browser test. It stops on failure. Both use synthetic data; tests force rules mode and do not prove live LLM quality.

- Python sample: Upload samples/practice-jd-SYNTHETIC.docx, then its PDF. Both should find Python/COD and one mandatory group. Wrong: no Python, extra unrelated skills or a made-up rating.
- Paste: Paste Required: Python and SQL. Expect two skills in one AND group. Compare the exact source words and save/reload; do not accept a success message alone.
- Different wording: Try a reconstructed data JD and Required: Python and either PyTorch or TensorFlow. Inspect groups; the OR pair must not become two compulsory skills.
- Bad or unknown input: Try an empty file, text renamed .pdf and Required: Apache Arrow. Expect a useful error or a visible review warning; never quietly claim complete coverage.
- Provider failure: Without consent, no model call is allowed. Mock tests cover bad quotes and provider failure. A fallback must say rules/unavailable; HTTP 200 alone does not prove LLM use.

Use an unfamiliar document too

Use only documents you own or have permission to process. Before running extraction, list expected skills and supporting words yourself. Compare missing and invented skills, sections, groups and quotes. A successful API response or valid JSON is not proof of correct extraction. Do not commit real CVs or send them to Groq without explicit permission.

If something fails

Read the FIRST error. Missing Python/npm: install and reopen the terminal. Missing packages: rerun setup. Browser cannot reach API: keep api running. Port busy: stop your own earlier server or coordinate a free port; do not kill unknown processes. Wrong extraction: compare the source, review and add a regression case.

7. Missing resources and what to do

What you can do now

All local teaching inputs are synthetic. No organizer pack, API key or cloud account is needed to run, edit and test your module. The tiny samples teach fields and behavior; the larger reconstructed pack remains under data/synthetic/. Neither is official organizer data.

Your samples - every file has a purpose

- expected.json - Small expected-result checklist; compare semantics, not generated IDs.
- input.json - Complete upload description or JSON request shown in this guide.
- output.json - Complete response captured from that actual API request.
- practice-jd-SYNTHETIC.docx - The same teaching JD in Word format.
- practice-jd-SYNTHETIC.pdf - Tiny invented Python JD for upload testing.

Groq: needed for live LLM verification, not offline practice

Use an authorized Groq account/key. If root .env does not exist, copy .env.example to .env; otherwise edit the existing private file carefully. Add these server-side values, restart api, choose hosted consent for ONE synthetic upload and check llm_status=processed plus source correctness. No LangChain wrapper is used; this implementation calls Groq directly.

```
RADIX_LLM_MODE=auto
RADIX_LLM_API_KEY=YOUR_PRIVATE_GROQ_KEY
RADIX_LLM_MODEL=openai/gpt-oss-20b
```

The placeholder is not a real key. Never put the key in frontend code, VITE_ variables, Git, a screenshot or an AI upload. If unavailable, continue rules-mode development and report live verification pending. If the model/key is unavailable, the configured fallback may return rules with llm_status=unavailable; do not call it model success.

8. Submit and explain your module

Completion checklist

[ ] I can run/explain it. [ ] Required inputs/outputs work. [ ] Normal/failure cases checked. [ ] Tests passed or failures reported. [ ] Contracts/source links preserved. [ ] No secrets/private CVs committed.

Push to YOUR own GitHub

Create an EMPTY private repository on your own account. From your full working clone, replace the URL below with YOUR new repository, review/stage only your changes, then run:

```
git remote rename origin scrum-master
git remote add origin https://github.com/YOU/YOUR-REPO.git
git add jd_analytics
.\.venv\Scripts\python.exe scripts/scan_handoff.py --staged
git commit -m "Verify my RADIX module"
git push -u origin main
```

Keep the whole working clone, including your source/tests/samples, guides, launcher and shared support. Never force-push. Grant the Scrum Master access if private.

Return URL, git rev-parse HEAD, changed files, actual test commands/results, three normal + three failure cases, synthetic screenshots and limitations. Distinguish implemented, mock-tested, live-verified and pending. The Scrum Master handles integration; you need not solve central merges.

A 45-second explanation for judges

I own the part that turns a job advert into requirements. It accepts a PDF, Word file or pasted text. Our existing Groq path can interpret prose when configured and permitted; local rules remain clearly labelled. Every proposed skill keeps source evidence. The user reviews importance and alternatives before saving, and matching uses that saved version. My tests check the actual words and groups, including failure cases. Live model quality and the official organizer pack are separate pending checks.

Three likely questions

- Why not just count keywords? Prose contains alternatives, negation and implied requirements. Rules are a limited fallback; the LLM path still needs a live quality test.
- Is confidence a skill level? No. It describes support for the extraction, not the candidate's ability.
- How does another module use it? Module 5 loads the reviewed JD by ID and revision using the shared fields.

Copy this prompt into your AI assistant

Help me complete module 1: JD Analytics step by step using this guide. First run it; request the exact section-5 source files and tests before editing. Preserve working code and shared fields. Fix a reproduced issue and test it. Keep confidence separate from competency and scenarios separate from saved profiles. Never invent official data or request pasted keys. Help me submit my own GitHub URL, tested commit, results and honest limitations.